Python Engineering - NumPy Quick Note

Anadir filas a NumPy arrays: append, vstack, concatenate y preallocation

Nota de apoyo para **week_02_data_structures -> class_07_numpy_arrays**. Objetivo: entender como hacer crecer una trayectoria durante un loop y por que NumPy no funciona igual que una lista de Python.

1. Diferencia fundamental: list.append() vs np.append()

Una lista de Python puede crecer dinamicamente. Un **ndarray** tiene un bloque de memoria de tamano fijo. Por eso `np.append()` no modifica el array original: crea uno nuevo.

```
import numpy as np

A = np.array([1, 2, 3])
np.append(A, 4)

print(A)
# [1 2 3]  <- A did NOT change

A = np.append(A, 4)
# now A is [1 2 3 4]
```

Idea clave: con NumPy normalmente debes reasignar el resultado: `A = np.append(A, ...)`.

2. Peligro: np.append() sin axis aplana

```
A = np.array([[0.0, 50.0, 0.0]])
new_state = np.array([5.0, 49.019, 0.1])

B = np.append(A, new_state)

# B becomes 1D:
# [0.0, 50.0, 0.0, 5.0, 49.019, 0.1]
```

Si omites `axis`, NumPy aplana los datos antes de concatenarlos. Para mantener una matriz de estados necesitamos trabajar por filas.

3. np.append(..., axis=0)

```
trajectory = np.array([[h0, v0, t0]])

new_state = np.array([[h_new, v_new, t_new]])

trajectory = np.append(
    trajectory,
    new_state,
    axis=0
)
```

Observa que `new_state` tiene forma `(1, 3)`, no `(3,)`. Para concatenar filas, el numero de columnas debe coincidir.

4. Convertir una fila 1D en fila 2D

```
new_state = np.array([h_new, v_new, t_new])

new_state_2d = new_state.reshape(1, 3)
# shape: (1, 3)

# equivalent:
new_state_2d = new_state[np.newaxis, :]
```

5. np.vstack(): muy legible para anadir filas

```
trajectory = np.array([[h0, v0, t0]])

new_state = np.array([h_new, v_new, t_new])
```

```
trajectory = np.vstack((trajectory, new_state))
```

`vstack` significa vertical stack: apilar verticalmente. Para un ejercicio como el nuestro es probablemente la opcion mas intuitiva si quieres hacer crecer una matriz fila a fila.

6. np.concatenate(): la opcion general

```
new_state = np.array([[h_new, v_new, t_new]])

trajectory = np.concatenate(
    (trajectory, new_state),
    axis=0
)
```

`np.concatenate()` es mas general: concatena arrays a lo largo del eje que indiques. `vstack()` es una forma mas expresiva para el caso concreto de apilar filas.

7. Que significa axis?

Operacion	Interpretacion
axis=0	crecer en filas; anadir estados debajo
axis=1	crecer en columnas; anadir nuevas variables al lado

```
# matrix shape: (N, 3)
# axis 0 -> N grows
# axis 1 -> 3 grows
```

8. Dentro de nuestro while de Euler

```
trajectory = np.array([[h0, v0, t0]])

while trajectory[-1, 0] >= 0:
    h, v, t = trajectory[-1, :]

    h_new, v_new, t_new = euler_step(h, v, t, g, dt)

    new_state = np.array([h_new, v_new, t_new])
    trajectory = np.vstack((trajectory, new_state))
```

Este codigo funciona y es perfecto para entender el concepto. Pero cada llamada a `vstack()` crea un array nuevo y copia los datos anteriores.

9. Opcion recomendada si NO sabemos cuantos pasos habra

Para una simulacion que termina cuando ocurre un evento (por ejemplo h < 0), normalmente no conocemos de antemano el numero exacto de filas. En ese caso, una lista Python sigue siendo una gran solucion:

```
trajectory = [(h0, v0, t0)]

while trajectory[-1][0] >= 0:
    h, v, t = trajectory[-1]
    h_new, v_new, t_new = euler_step(h, v, t, g, dt)
    trajectory.append((h_new, v_new, t_new))

trajectory = np.array(trajectory)
```

Esta suele ser la estrategia mas eficiente y limpia: list.append() durante la fase dinamica, np.array() cuando ya tenemos todos los estados y queremos analizarlos numericamente.

10. Opcion recomendada si SI conocemos N

```
N = 1000
trajectory = np.empty((N, 3))

trajectory[0, :] = [h0, v0, t0]

for i in range(1, N):
```

```
h, v, t = trajectory[i-1, :]
h_new, v_new, t_new = euler_step(h, v, t, g, dt)
trajectory[i, :] = [h_new, v_new, t_new]
```

Esto se llama **preallocation**: reservamos toda la memoria una sola vez y despues rellenamos filas. Para simulaciones grandes suele ser mucho mas eficiente.

11. Comparacion rapida

Metodo	Funciona en loop?	Eficiencia	Recomendacion
list.append() + np.array()	Si	Alta	Mejor si N es desconocido
np.vstack()	Si	Media/baja	Muy claro para aprender
np.append(axis=0)	Si	Media/baja	Valido, cuidado con shapes
np.concatenate()	Si	Media/baja	General y flexible
np.empty()/zeros()	Si	Alta	Mejor si N es conocido

12. Mini cheat sheet

```
np.append(A, row, axis=0)      append along rows
np.vstack((A, row))            stack row below
np.concatenate((A,B), axis=0)  general concatenation
row.reshape(1, -1)             convert 1D row to 2D
row[np.newaxis, :]             same idea
np.empty((N, 3))               preallocate memory
A[i, :] = row                  fill existing row
```